第27课 前缀和与差分入门

一、课程信息

项目	内容
课次	第 27 课 / 共 40 课
主题	前缀和与差分入门
前置知识	一维数组、for 循环、累加求和
后续衔接	第 28 课——二分查找

教学目标（可检验）

1. **能手动构建**：给定一个长度为 n 的数组，学生能在纸上正确写出对应的前缀和数组。
2. **能编程求区间和**：用前缀和数组，在 O(1) 时间内求出任意区间 [l, r] 的元素之和，并通过至少一道完整题目。
3. **能理解差分思想**：知道差分是前缀和的逆操作，能用差分数组完成"区间内所有元素加上同一个数"的操作。

二、教学重点与难点

重点

- 前缀和数组的含义：`sum[i] = a[1] + a[2] + ... + a[i]`。
- 区间和公式：`区间 [l, r] 的和 = sum[r] - sum[l-1]`。
- 构建前缀和数组的代码写法。

难点

- **下标从 1 开始**以及 `sum[0] = 0` 这个"哨兵"设计——学生容易和从 0 开始的数组搞混。
- **区间和公式中 `l-1`** 的理解——为什么要减去 `sum[l-1]` 而不是 `sum[l]`。
- **差分的概念**——"逆操作"比较抽象，需要大量具体数字辅助。

三、课堂时间分配（共 100 分钟）

时间	时长	环节	说明
0′–5′	5 min	热身回顾	用一道"暴力求区间和"引出痛点
5′–25′	20 min	讲解 ① 前缀和概念与公式	比喻 + 板书画图 + 手动模拟
25′–40′	15 min	例题 1 + 例题 2（边讲边练）	学生同步敲代码
40′–50′	10 min	课堂练习 1、2	学生独立完成，教师巡视
50′–55′	5 min	休息 / 答疑	
55′–75′	20 min	讲解 ② 差分概念与用法	比喻 + 板书画图 + 手动模拟
75′–85′	10 min	例题 3（差分）	边讲边练
85′–95′	10 min	课堂练习 3、4	学生独立完成
95′–100′	5 min	总结 + 作业布置 + 下节课预告	

> **注**：两段讲解分别为 20 分钟和 20 分钟，均未超过 25 分钟上限。

四、讲稿提纲

第一部分：前缀和（20 分钟）

4.1 热身——"笨办法"有多慢？

> 同学们好！今天我们来聊一个"偷懒的好办法"。
>
> 先想一个场景：**体育课跑圈**。
>
> 小明每天跑步，教练记录了他每一圈跑了多少米：
>
> | 圈数 i | 1 | 2 | 3 | 4 | 5 |
> |---|---|---|---|---|---|
> | 每圈距离 a[i] | 400 | 350 | 380 | 420 | 390 |
>
> 如果教练问：**"第 2 圈到第 4 圈一共跑了多少米？"**
>
> 你会怎么算？—— `350 + 380 + 420 = 1150`，要把三个数加起来。
>
> 如果教练一天问 100 次不同的区间呢？每次都从头加，好累呀！
>
> 有没有更聪明的办法？——**有！就是前缀和。**

4.2 比喻引入——存折余额

> 想象你有一个**存折**，每个月存一笔钱：
>
> | 月份 | 1 月 | 2 月 | 3 月 | 4 月 | 5 月 |
> |---|---|---|---|---|---|
> | 存入 a[i] | 100 | 200 | 150 | 300 | 250 |
> | 余额 sum[i] | 100 | 300 | 450 | 750 | 1000 |
>
> **余额 = 从第 1 个月到现在所有存款的总和。**
> 这就是**前缀和**！ `sum[i]` 就是"到第 i 个位置为止的总和"。

在白板上写出公式：

```
sum[0] = 0            （还没存钱，余额为 0）
sum[1] = a[1]
sum[2] = a[1] + a[2]
sum[i] = a[1] + a[2] + ... + a[i]
```

递推写法（关键！）：

```
sum[i] = sum[i-1] + a[i]
```

> 翻译成大白话：**今天的余额 = 昨天的余额 + 今天存的钱。**

4.3 区间和公式——"减法的魔法"

> 现在教练问：**"第 2 圈到第 4 圈一共跑了多少米？"**
> 我们已经有了前缀和数组（就是余额）：
>
> | i | 0 | 1 | 2 | 3 | 4 | 5 |
> |---|---|---|---|---|---|---|
> | sum[i] | 0 | 400 | 750 | 1130 | 1550 | 1940 |
>
> 第 2 圈到第 4 圈的总和 = **前 4 圈的总和 − 前 1 圈的总和** = `sum[4] - sum[1]` = 1550 − 400 = **1150**
> 写成公式：**区间 [l, r] 的和 = sum[r] − sum[l−1]**

板书画图（重点！画一条数轴，标出区间，用颜色标出"被减掉"的部分）：

```
a:       [  a1  |  a2  |  a3  |  a4  |  a5  ]
sum[4]: [===========总共这么多============]
sum[1]: [=这部分=]
差值:            [======我们要的部分======]
```

> 就像量身高：你站在第 4 级台阶上，减去第 1 级台阶的高度，就是第 2 到第 4 级台阶的高度差！

4.4 代码实现

在白板上写出两步代码模板：

第一步：构建前缀和

```
for (int i = 1; i <= n; i++) {
    sum[i] = sum[i-1] + a[i];
}
```

第二步：回答查询

```
cout << sum[r] - sum[l-1] << endl;
```

> **特别提醒**：我们让数组从 **下标 1** 开始存，`sum[0] = 0`，这样 `l = 1` 时，`sum[l-1] = sum[0] = 0`，不会出错！

第二部分：差分入门（20 分钟）

4.5 引入——一个新问题

> 故事继续：学校举办"阅读马拉松"活动。
> 有 5 位同学，一开始每人的积分都是 0：
>
> | 编号 | 1 | 2 | 3 | 4 | 5 |
> |---|---|---|---|---|---|
> | 积分 | 0 | 0 | 0 | 0 | 0 |
>
> 老师说："**第 2 号到第 4 号同学每人加 5 分！**"
> 又说："**第 1 号到第 3 号同学每人加 3 分！**"
> 如果每次都用 for 循环一个一个加，操作 m 次、每次加 n 个人，时间要 n × m，太慢了！
> 有没有偷懒的办法？——**有！就是差分。**

4.6 差分的概念

> **差分是前缀和的"反义词"**（逆操作）。
> 前缀和：每天的余额 = 从第一天到今天的**总收入**
> 差分：每天的**收入变化** = 今天比昨天多赚了多少
> 给定数组 `a`，它的差分数组 `d` 定义为：
> `d[1] = a[1] d[i] = a[i] - a[i-1] （i >= 2）`
> **重要关系**：对差分数组 `d` 做前缀和，就能还原回原数组 `a`！

板书用具体数字演示：

```
原数组 a:   2   5   3   7   6
差分 d:     2   3  -2   4  -1

验证：对 d 做前缀和
  d[1] = 2                a[1] = 2
  d[1]+d[2] = 2+3 = 5     a[2] = 5
  2+3+(-2) = 3            a[3] = 3
  2+3+(-2)+4 = 7          a[4] = 7
  2+3+(-2)+4+(-1) = 6     a[5] = 6
```

4.7 差分的妙用——区间加

> 回到"加分"的问题：要让 **a[l] 到 a[r] 都加上 v**，
> 在差分数组里只需要做 **两步**：
> `d[l] += v; // 从第 l 个开始，每个人都多了 v d[r+1] -= v; // 从第 r+1 个开始，取消这个加法`
> 最后对差分数组做一遍前缀和，就得到最终结果！

板书画图——像"开水龙头"和"关水龙头"：

> 想象一排水桶（编号 1 到 5），你要给第 2 到第 4 个水桶各加 5 升水。
> - 在第 2 个水桶的位置**打开水龙头**（+5）。
> - 在第 5 个水桶的位置**关掉水龙头**（-5）。
> - 水从左往右流过去（做前缀和），中间的水桶就都装了 5 升水！

```
差分数组:   0  +5   0   0  -5   0
前缀和后:   0   5   5   5   0   0

              开水龙头      关水龙头
```

七、常见错误与应对

错误 1：下标从 0 开始，忘记设 sum[0] = 0

```
//  错误：数组下标从 0 开始，公式会乱
for (int i = 0; i < n; i++) {
    cin >> a[i];
    sum[i] = sum[i-1] + a[i];  // i=0 时 sum[-1] 越界!
}
```

应对：强调本课统一用**下标从 1 开始**，`sum[0] = 0` 是"地基"。在黑板上画一个"第 0 格"提醒学生。

错误 2：区间和公式写成 sum[r] - sum[l]

```
//  错误：少减了一个
cout << sum[r] - sum[l] << endl;  // 漏掉了 a[l]！
```

应对：用具体数字验证。比如求 [2, 4] 的和：
- 正确：`sum[4] - sum[1]` = (a1+a2+a3+a4) - (a1) = a2+a3+a4
- 错误：`sum[4] - sum[2]` = (a1+a2+a3+a4) - (a1+a2) = a3+a4 （少了 a2！）

记忆口诀：`sum[r] - sum[l 减 1]`，**"左边减一"**。

错误 3：前缀和没用 long long，导致溢出

```
//  错误：n=100000 且每个数是 10000，总和可达 10 亿，int 可能溢出
int sum[100005];
```

应对：告诉学生一个简单的判断方法——如果 n × 最大值 > 20 亿（int 的上限约 21 亿），就用 `long long`。前缀和题目**建议一律用 `long long`**。

错误 4：差分忘记在 r+1 处减回来

```
//  错误：只加了开头，没关水龙头
d[l] += v;
// 忘了 d[r+1] -= v;
```

应对：回忆"水龙头"比喻——开了水龙头一定要关！否则后面的水桶全都会被灌满。让学生养成习惯：`d[l] += v` 和 `d[r+1] -= v` **永远成对出现**。

错误 5：差分中 r+1 越界

```
// 当 r == n 时，d[r+1] = d[n+1]
// 如果数组开小了就越界！
```

应对：数组多开几个位置（例如 `d[100005]` 而不是 `d[100000]`），养成多开的好习惯。

九、下节课预告

> 同学们，这节课我们学了两个"偷懒神器"——**前缀和**帮我们快速求区间和，**差分**帮我们快速做区间加。
>
> 下节课（第 28 课），我们要学另一个超级实用的算法——**二分查找**！
>
> 想象你在一本 1000 页的字典里找一个字，你会从第 1 页翻到第 1000 页吗？当然不会！你会先翻到中间，看看大了还是小了，再决定往前翻还是往后翻。这个"猜数字"的思路，就是二分查找的核心。下节课见！

> **教师备注**：本课内容偏难，是学生第一次接触"用空间换时间"的预处理思想。课堂上务必多画图、多用具体数字手动模拟。如果学生对差分掌握不牢，可以课后再安排一次差分专项小练习，不必强求一节课全部消化。